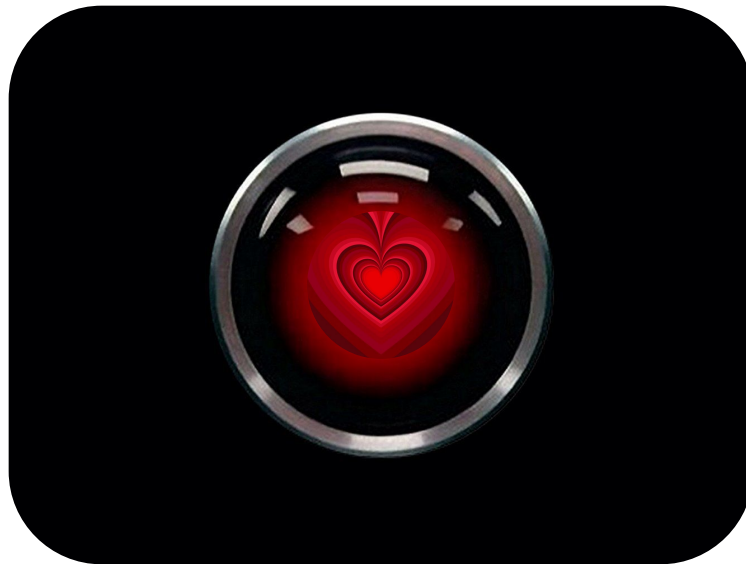


LLM-assisted Coding (in Python)

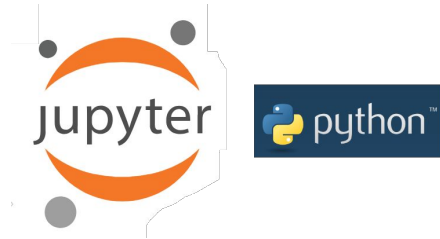
for Data Analysis Pipelines in Neuroscience



Andrew Savoy
Postdoctoral Fellow
Smiley Lab

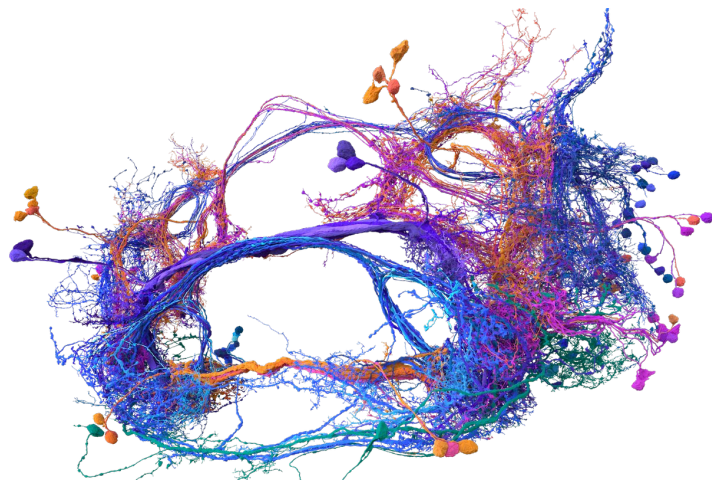
Hello! While we wait to begin, please do this:

1. Sign in to your (**UMich**) **Google account**
2. Go to **colab.research.google.com**
3. Click **New notebook**
4. In the empty cell, type: `print("hello")`
5. Press **Shift + Return** (or **Shift + Enter**)
6. `hello` should appear below the cell
7. Open **your preferred AI/LLM interface** in a new browser tab or window (or elsewhere).
Alternatively, you may use the built-in AI assistant in Colab.



Things I know you know

- **Computers are useful in science**
 - **Speed, precision, complexity, big data, automation**
 - Counting cells in histological images
 - Bootstrapping in statistical analysis
 - Detecting spikes and neuronal units in ephys data
 - Reconstructing neurons from electron microscopy scans
- **Custom code is SUPER useful**

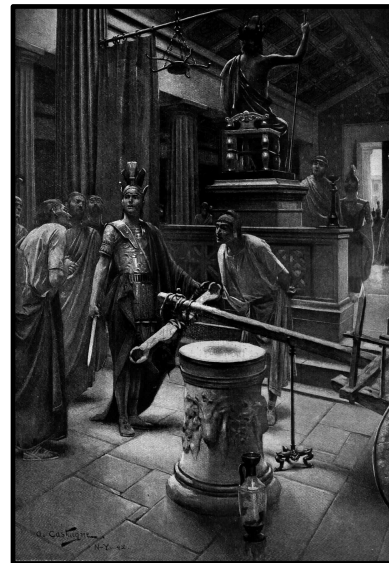


How do you create your own custom code?

- Maybe don't? It might be fine to use point-and-click software (e.g., SPSS)
- Copy code you find online (e.g., Github, Stack Overflow, etc.)
- Have someone else write it for you
- Write it with someone else
- Write it yourself
- Some combination of the above

But now you can...

- Ask an LLM to generate code for you



Which word has the most Vs in it

Before I show you Claude's response, think about how you would figure this out.

Write a message...



Sonnet 4.6 Max ▾



Claude is AI and can make mistakes. Please double-check responses.

Which word has the most Vs in it

Revivivivify is sometimes cited, but it's not a real word.

The legitimate English word with the most V's is likely **vivivivorous** — also not real.

Among actual dictionary words, **vivify** and **revivify** each contain two V's, and **revivification** contains three. That's the highest count I'm confident about for standard English.



Write a message...



Sonnet 4.6 Max ▾



Claude is AI and can make mistakes. Please double-check responses.

Use Python code to answer the question.

Excavated lexical constraints and identified maximum vowel threshold

⌚ Architected Python solution to identify word with maximum V occurrences.

⌚ Count V's in each word from system dictionary ▾

```
python3 -c '
with open('/usr/share/dict/words') as f:
    words = f.read().splitlines()

scored = [(w.lower().count('v'), w) for w in words]
scored.sort(reverse=True)

for count, word in scored[:10]:
    print(f'{count} v\\\'s: {word}')
```

⌚ Excavated lexical constraints and identified maximum vowel threshold.

✓ Done

The system dictionary caps out at **2 V's**. The top words are **vuvuzela**, **vulva**, and **votive** — tied at two.



I know for a fact that there are real words with 3 V's. Write better code to find them.

Methodically examined words, recounted letter frequencies, reconsidered approach

The system dictionary is too small. Let me use a larger corpus.

Excavated maximum V-count words and validated findings

Several real words hit 3 V's. The most recognizable are **ovoviviparous** (embryos that develop inside eggs retained within the parent) and **vivisection**, along with the medical terms **vulvovaginal** and **vulvovaginitis**. Nothing in the NLTK corpus exceeds 3.

Show me your code.

Orchestrated clean code presentation

python

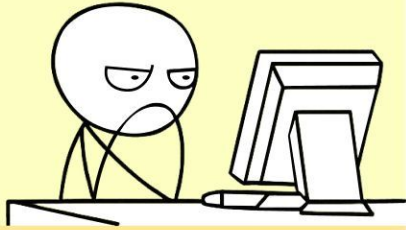
```
import nltk
nltk.download('words', quiet=True)
from nltk.corpus import words

found = [(w.lower().count('v'), w) for w in words.words()]
found = [(c, w) for c, w in found if c >= 3]
found.sort(reverse=True)

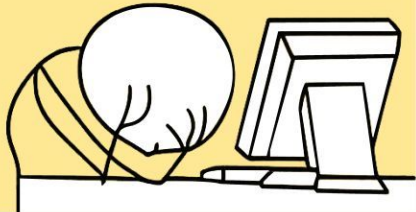
for count, word in found:
    print(f"{count} v's: {word}")
```


Before AI

Write new analysis pipeline in 2 hours

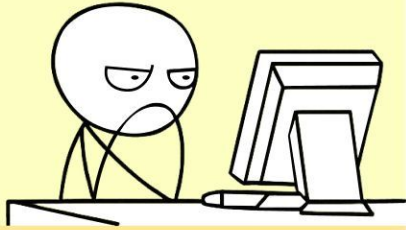


Debug it for 4 hours

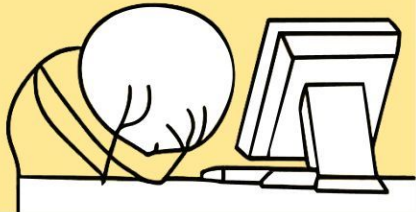


Before AI

Write new analysis pipeline in 2 hours



Debug it for 4 hours

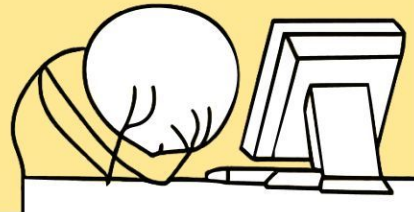


After AI

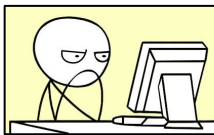
Write new analysis pipeline in 5 minutes



Debug it for 24 hours

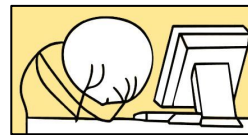


Common Minor Problems



- **Buggy code**
 - Lots of little things don't quite work
- **Bad inference**
 - Usually due to bad prompt writing
 - Or it forgot the larger context, maybe from lossy compression of the session text
 - It can get stuck in a cycle of misunderstanding
- **It overwrites code you wanted**

Common Major Problems



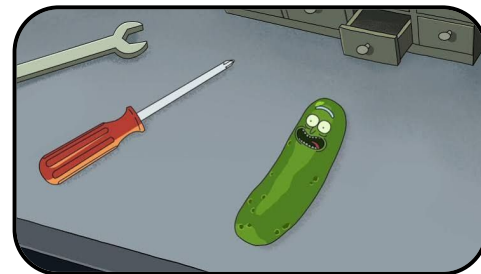
- **Right analysis, wrong data, but you are unaware**
 - **Even worse: it created fake data and didn't tell you**
- ***Wrong analysis*, but you are unaware**
 - **Even worse: your published paper says you did the right analysis, but you didn't**
- **Technically nothing is “wrong,” but methods are extremely weird, atypical, stupid, etc., and you are unaware**

Guidelines—Before you generate code

1. **Think through your problem and how to solve it with code**
 - a. Know your file structure, data structure, goal, appropriate methods, etc.
2. **Choose the right LLM model for your needs**
 - a. Can it handle the size of the task?
 - b. It is intended for coding?
 - c. Can it access tools and run separate agents?
3. **Before generating any code, decide if you want your LLM to have access to your prior sessions (memory)**
4. **Breakdown your intended pipeline into modules, then code one module at a time**
 - a. In a Colab/Jupyter notebook (.ipynb file), each module can be in a cell, or each module can be its own standalone script (.py file)
5. **Give precise, detailed instructions in your prompt**
 - a. Describe the input data in detail
 - b. Describe the desired output in detail
6. **Ask it to plan what it will do before it does it**
 - a. Ask it to ask you questions to help it improve the plan
 - b. Give it feedback on the plan
7. **Know how to assess the quality of the output**
 - a. Do you know what “wrong” would look like? If not, figure that out before viewing the output



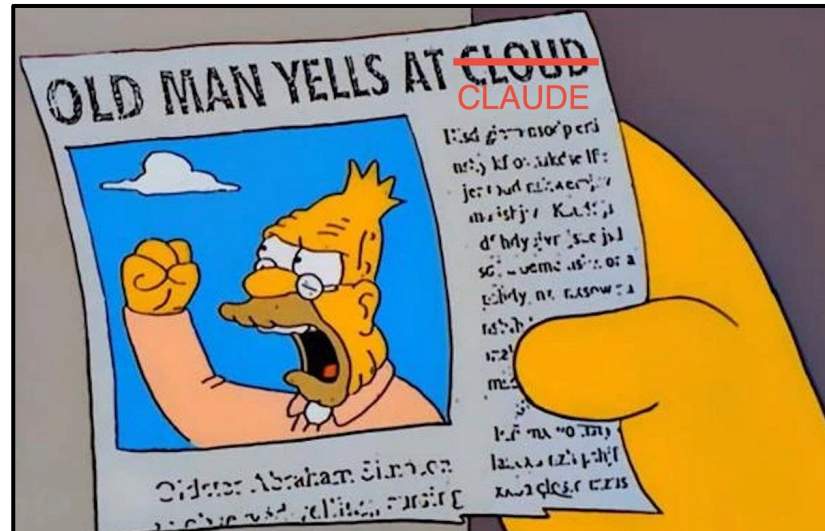
Know your tool, don't be a tool!



Don't subtract yourself from the science

Guidelines—After you generate code

1. Understand what it did
 - a. Ask it to explain what it did and study until you understand
2. If you get a traceback (error message) after running the code, put the entire traceback into a prompt
3. Share your code and data when you publish



If you haven't already, please do this:

1. Sign in to your (**UMich**) **Google account**
2. Go to **colab.research.google.com**
3. Click **New notebook**
4. In the empty cell, type: `print("hello")`
5. Press **Shift + Return** (or **Shift + Enter**)
6. `hello` should appear below the cell
7. Open **your preferred AI/LLM interface** in a new browser tab or window (or elsewhere).
Alternatively, you may use the built-in AI assistant in Colab.

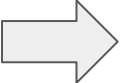




[4]
✓ 0s
▶ print("hello")
... hello



Get the data:

Go to  tinyurl.com/2s39ffzx

1. In a new cell, copy, paste, and run:

```
!git clone https://github.com/a-savoy/hvc-song-response-exercise.git  
%cd hvc-song-response-exercise
```

2. In a new cell, copy, paste, and run:

```
from IPython.display import Image  
Image("target_figure.png", width=600)
```

3. In a new cell, copy, paste, and run:

```
df = pd.read_csv("data/unfamiliar_spikes.csv")  
df
```

4. This will produce an error! Try to solve it... somehow...

 **NameError:**



Files



..

▶ .config

▶ .git

▶ data

.gitignore

LICENSE

[m] README.md

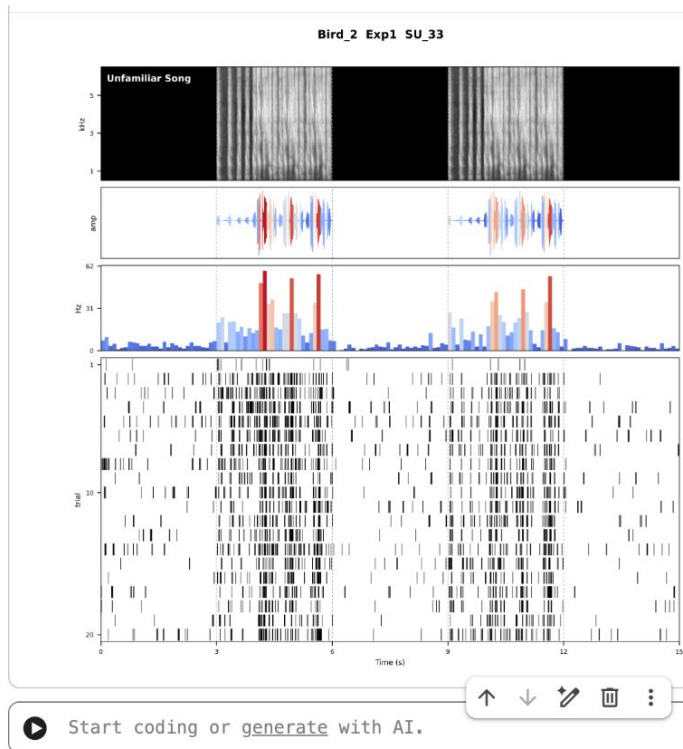
reference_solution.ipynb

requirements.txt

target_figure.png

▶ sample_data

Disk 87.51 GB available



README.md



Female zebra finch HVC single-neuron song-response exercise

A small, self-contained Python data-analysis exercise. Starting from real recordings, reproduce the target figure, which shows spiking from one neuron in female zebra finch HVC (a song-related brain area), in response to an unfamiliar male's song. Each trial plays the same 3 s song twice (a "tandem pair": song, gap, song).

Setup Google Colab

You'll do all of this in a Colab notebook. There is nothing to install on your own computer.

1. Sign in to your Google account (use your UMich account).
2. Go to colab.research.google.com.
3. Click New notebook.
4. In the empty cell, type: `'print("hello")'`
5. Press Shift + Return (or Shift + Enter).
6. 'hello' should appear below the cell.

Then open your preferred AI/LLM interface in another browser tab or window, or use Colab's built-in AI assistant. See [Using an AI assistant](#) below.

Get the data into your notebook

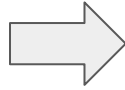
In a new cell, run:

```
!git clone https://github.com/a-savoy/hvc-song-response-exercise.git
%cd hvc-song-response-exercise
```

That copies this repository into your Colab session and moves you

Your task:

Use the data provided to create exactly this figure



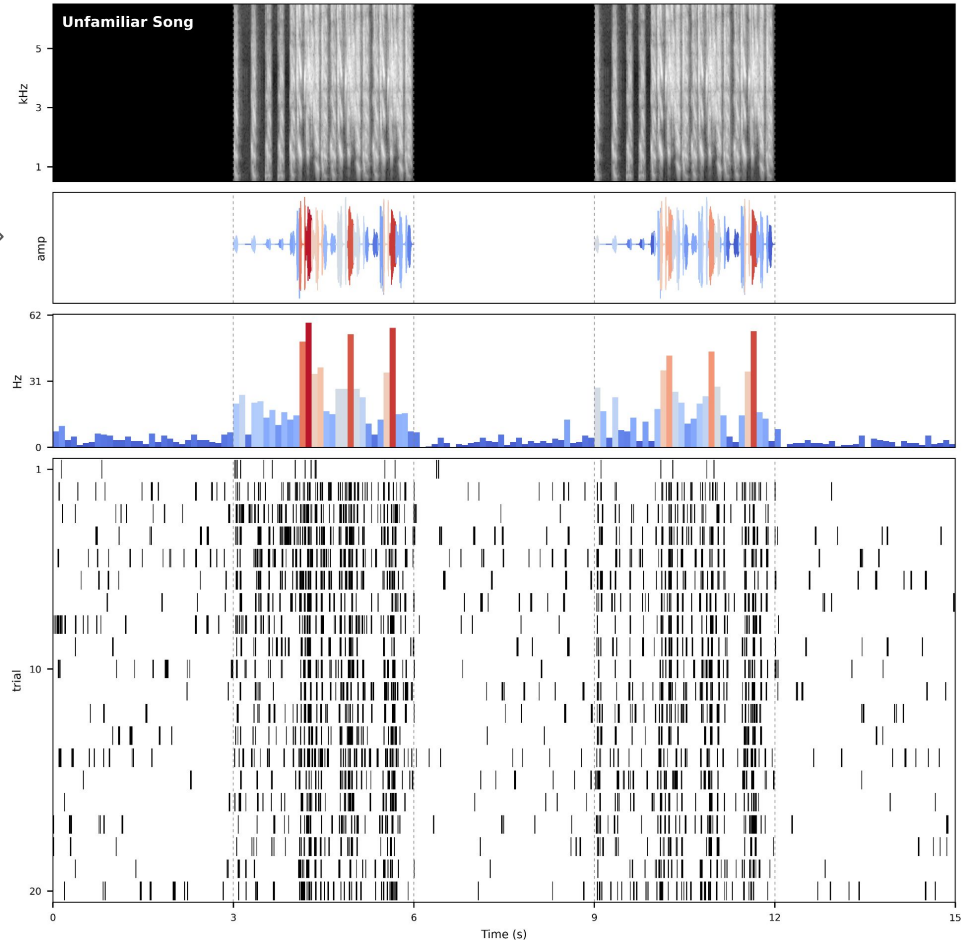
You may use AI, but...

...don't show it this figure!

...follow the guidelines I gave you

...help each other

...don't look at my code until the end



Alternatively, use Jupyter Notebook:



If you have Windows:

1. Open **Command Prompt** and enter:

```
python --version
```

2. If Python is missing, enter:

```
winget install --id Python.Python.3.14
```

3. Close and reopen **Command Prompt**; enter:

```
python --version
```

4. Enter: `python -m pip install notebook`

5. Enter: `python -m jupyter notebook`



If you have macOS:

1. Open **Terminal** and enter:

```
python3 --version
```

2. If Python is missing, enter:

```
xcode-select --install
```

3. Close and reopen **Terminal** and enter:

```
python3 --version
```

4. Enter: `python3 -m pip install --user notebook`

5. Enter: `python3 -m jupyter notebook`

These lines are the same length:



Statistics said so. AI agreed.